ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №5

«Обробка помилок, файли та серіалізація»

з дисципліни «Професійний Python»

Виконав: студент групи ФеП-33с

Цястун Павло

Варіант 1

Перевірив: професор Свелеба Сергій Андрійович

Львів – 2026

1. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних.

2. Номер варіанта та постановка задачі

Варіант 1. Імпорт та експорт даних системи обліку студентів.

Необхідно створити надійний модуль імпорту, експорту та конфігурації, який читає дані студентів із CSV-файла, перевіряє їх, пропускає або реєструє некоректні записи залежно від політики, експортує коректні записи у JSON та не залишає пошкоджений результат у разі помилки. Для варіанта 1 реалізовано:

- потоковий (streaming) імпорт CSV;
- перевірку ID, імені, прізвища та групи;
- перевірку average_grade у межах 0–100;
- режими strict та tolerant;
- експорт у JSON;
- YAML-конфігурацію з мінімальним середнім балом;
- статистику valid/invalid;
- logging некоректних записів.

Додатково реалізовано завдання підвищеної складності: єдині Protocol-інтерфейси importer та exporter, автоматичний вибір формату за розширенням (CSV та JSON Lines на вході, JSON та JSON Lines на виході), окремий CSV-звіт про помилки, logging із ротацією файлів, override конфігурації змінними середовища, atomic export, резервне копіювання існуючого результату (.bak), summary report, а також context manager для transaction-like запису файлів.

3. Схема вхідних даних

Вхідний CSV-файл має заголовок і п’ять полів (відповідно до варіанта 1):

Поле	Тип	Правило перевірки
student_id	ціле число	ціле, додатне
first_name	рядок	непорожнє, лише літери, дефіс, апостроф і пробіл
last_name	рядок	непорожнє, лише літери, дефіс, апостроф і пробіл
group	рядок	непорожня, без пробілів, довжина до 20 символів

Поле	Тип	Правило перевірки
average_grade	число	скінченне число від 0 до 100 (допускається десяткова кома)

Приклад data/students.csv. Файл містить 9 коректних записів і 5 некоректних (порожнє ім’я, оцінка 105, нечислова оцінка, нечисловий ID, порожня група):

```
student_id,first_name,last_name,group,average_grade
1,Pavlo,Tsiastun,FeP-33s,93.4
2,Olena,Savchuk,FeP-33s,87.1
3,Taras,Hrytsenko,FeP-31,71.8
4,Sofiia,Lysenko,FeP-32,96.0
5,Maksym,Kravets,FeP-31,54.5
6,Kateryna,Moroz,FeP-33s,78.9
7,,Bilyk,FeP-32,82.3
8,Yulia,Pavlenko,FeP-31,105
9,Bohdan,Rudenko,FeP-32,abc
x,Iryna,Kovalenko,FeP-33s,85.6
11,Andrii,Shevchuk,FeP-34,88.8
12,Mariia,Boyko,,79.3
13,Oksana,Melnyk,FeP-34,66.0
14,Ihor,Danylenko,FeP-34,59.9
```

4. Структура проєкту

```
reliable_importer_lab/
├── pyproject.toml
├── config.yaml
├── README.md
├── data/
│   └── students.csv
├── output/
├── logs/
├── src/
│   └── reliable_importer/
│       ├── __init__.py
│       ├── main.py
│       ├── config.py
│       ├── exceptions.py
│       ├── models.py
│       ├── readers.py
│       ├── validators.py
│       ├── exporters.py
│       ├── logging_config.py
│       ├── services.py
│       ├── utils.py
│       └── experiments.py
└── tests/
    └── test_importer.py
```

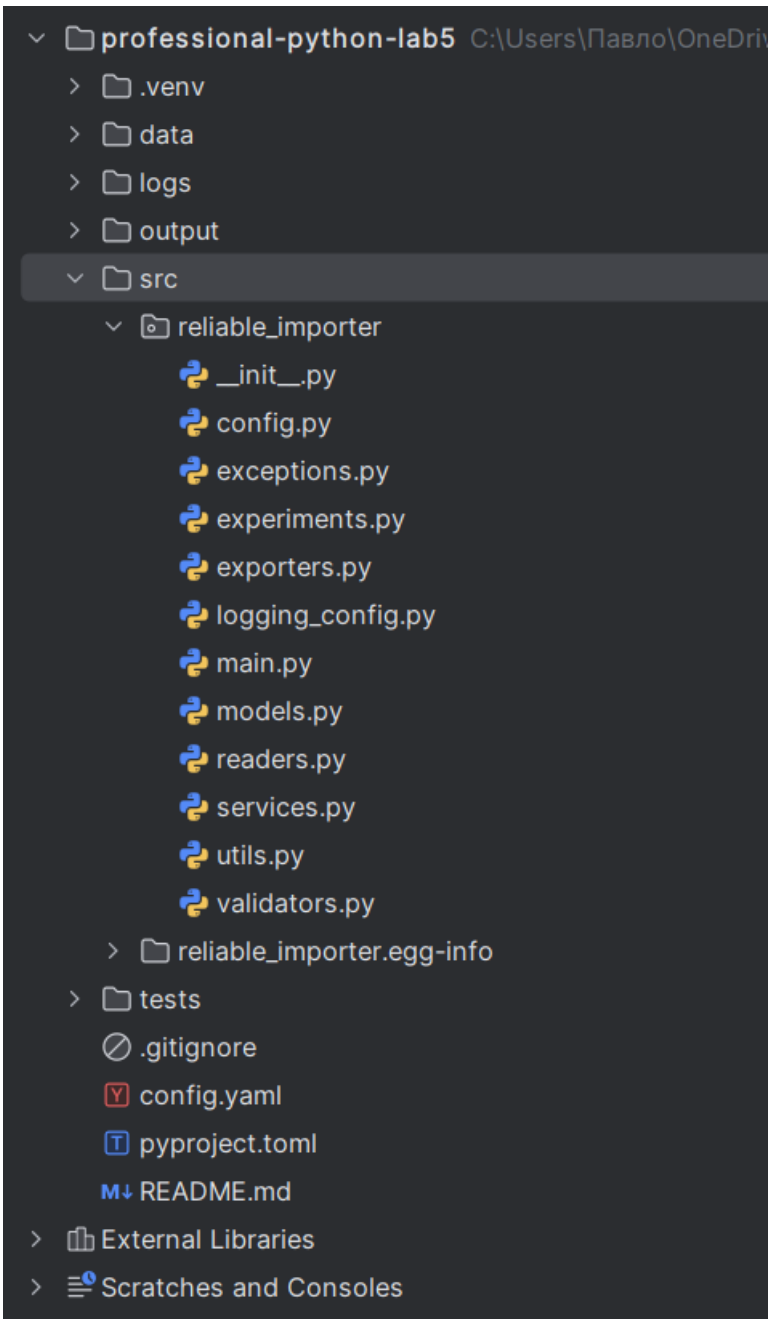


Рис. 1. Структура проєкту в PyCharm

5. Схема pipeline

```
config.yaml
    ↓
load_config()  →  validate_config()        (fail-fast)
    ↓
Input CSV
    ↓
read_csv_rows()          — streaming reader (один рядок за раз)
    ↓
parse_student()          — parsing + validation
    ↓
 valid ──────────────→  Student (domain object)
 invalid → RecordValidationError → logging
              ↙                        ↘
     tolerant: skip, +1 invalid     strict: abort (exit code 1)
    ↓
filter_by_grade()        — відповідно до minimum_grade з YAML
```

```
    ↓
JsonExporter              — streaming JSON у <name>.tmp
    ↓
atomic replace            — .tmp → students.json
    ↓
Logging / Report          — Total, valid, invalid, exported
```

6. Ієрархія exceptions

Усі помилки застосунку успадковуються від ApplicationError, тому main.py може перехоплювати їх одним специфічним блоком, не ховаючи програмних помилок (Exception обробляється окремо як неочікуваний збій). Гілка ConfigurationError описує помилки конфігурації, гілка DataError — помилки даних: DataImportError (не вдалося прочитати вхід), DataExportError (не вдалося записати вихід) і RecordValidationError (один запис порушує правило). RecordValidationError зберігає додаткові дані: номер рядка (line_number) і поле (field), що дозволяє точно вказати місце помилки в log та в error report.

```
ApplicationError
├── ConfigurationError
└── DataError
    ├── DataImportError
    ├── DataExportError
    └── RecordValidationError   (line_number, field)
```

```
"""Hierarchy of application exceptions."""



class ApplicationError(Exception):
    """Base application exception."""



class ConfigurationError(ApplicationError):
    """Configuration is missing or invalid."""



class DataError(ApplicationError):
    """Base class of data-related errors."""



class DataImportError(DataError):
    """Input data cannot be read."""



class DataExportError(DataError):
    """Output cannot be written."""
```

```
class RecordValidationError(DataError):
    """One record does not satisfy a validation rule."""


    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field
```

7. Exception chaining

Низькорівневі винятки (ValueError, KeyError, FileNotFoundError, yaml.YAMLError, OSError) перетворюються на доменні винятки конструкцією raise ... from error, тому причина не втрачається. У validators.py помилка перетворення float(...) стає RecordValidationError із збереженим __cause__ = ValueError:

```
try:
    grade = float(row["average_grade"].replace(",", "."))
except (KeyError, TypeError, ValueError, AttributeError) as error:
    raise RecordValidationError(
        "average_grade must be numeric",
        line_number=line_number,
        field="average_grade",
    ) from error
```

Те саме застосовано в config.py (FileNotFoundError → ConfigurationError), readers.py (OSError, UnicodeDecodeError, JSONDecodeError → DataImportError) та utils.py (OSError → DataExportError). Результат експерименту наведено в розділі 14.3.

8. Конфігурація

8.1. config.yaml

```
input:
  path: data/students.csv


output:
  path: output/students.json
  errors_path: output/errors.csv


processing:
  minimum_grade: 60
  skip_invalid: true


logging:
  level: INFO
```

```
  path: logs/application.log
```

Параметри: input.path — вхідний файл; output.path — результат (формат вибирається за розширенням); output.errors_path — окремий CSV-звіт про некоректні записи (необов'язковий); processing.minimum_grade — поріг фільтрації; processing.skip_invalid — політика (true — tolerant, false — strict); logging.level та logging.path — налаштування log. Змінити поведінку можна без редагування Python-коду.

8.2. Завантаження та валідація

Файл читається через yaml.safe_load (безпечний для недовірених даних). Помилки відсутнього файла, некоректного YAML, відсутнього ключа та неправильних типів перетворюються на ConfigurationError із збереженням причини. Окремо виконується fail-fast перевірка перед початком обробки: minimum_grade у межах 0–100, допустимий рівень logging, існування вхідного файла та те, що output.path не є каталогом. Додатково реалізовано override параметрів змінними середовища IMPORTER_MINIMUM_GRADE, IMPORTER_SKIP_INVALID та IMPORTER_LOG_LEVEL (вони мають пріоритет над YAML).

```
"""Loading, environment override and validation of the YAML configuration."""


import os
from collections.abc import Mapping
from pathlib import Path
from typing import Any


import yaml


from reliable_importer.exceptions import ConfigurationError
from reliable_importer.models import AppConfig, LoggingConfig, ProcessingConfig


LOG_LEVELS = ("DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL")


ENV_MINIMUM_GRADE = "IMPORTER_MINIMUM_GRADE"
ENV_SKIP_INVALID = "IMPORTER_SKIP_INVALID"
ENV_LOG_LEVEL = "IMPORTER_LOG_LEVEL"



def _read_yaml(path: Path) -> Any:
    try:
        with path.open("r", encoding="utf-8") as file:
            return yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Configuration file not found: {path}") from
error
    except yaml.YAMLError as error:
        raise ConfigurationError(f"Invalid YAML in {path}") from error
```

```
    except OSError as error:
        raise ConfigurationError(f"Cannot read configuration file: {path}") from
error



def _parse_bool(value: object, name: str) -> bool:
    if isinstance(value, bool):
        return value
    if isinstance(value, str) and value.lower() in ("true", "false", "1", "0"):
        return value.lower() in ("true", "1")
    raise ConfigurationError(f"{name} must be true or false, got {value!r}")



def _parse_number(value: object, name: str) -> float:
    if isinstance(value, bool) or not isinstance(value, (int, float, str)):
        raise ConfigurationError(f"{name} must be a number, got {value!r}")
    try:
        return float(value)
    except ValueError as error:
        raise ConfigurationError(f"{name} must be a number, got {value!r}") from
error



def build_config(
    raw: Any, environ: Mapping[str, str] | None = None
) -> AppConfig:
    """Convert a raw YAML structure into AppConfig (environment variables win)."""
    env = os.environ if environ is None else environ

    if not isinstance(raw, dict):
        raise ConfigurationError("Configuration must be a mapping at the top
level")

    try:
        input_path = Path(raw["input"]["path"])
        output_path = Path(raw["output"]["path"])
        errors_raw = raw["output"].get("errors_path")
        minimum = env.get(ENV_MINIMUM_GRADE, raw["processing"]["minimum_grade"])
        skip = env.get(ENV_SKIP_INVALID, raw["processing"]["skip_invalid"])
        level = env.get(ENV_LOG_LEVEL, raw["logging"]["level"])

        config = AppConfig(
            input_path=input_path,
            output_path=output_path,
            errors_path=Path(errors_raw) if errors_raw else None,
            processing=ProcessingConfig(
                minimum_grade=_parse_number(minimum, "processing.minimum_grade"),
                skip_invalid=_parse_bool(skip, "processing.skip_invalid"),
            ),
            logging=LoggingConfig(
                level=str(level).upper(),
                path=Path(raw["logging"]["path"]),
            ),
        )
    except KeyError as error:
        raise ConfigurationError(f"Missing configuration key: {error}") from error
    except (TypeError, AttributeError) as error:
        raise ConfigurationError("Configuration structure is invalid") from error

    return config
```

```
def validate_config(config: AppConfig) -> None:
    """Fail-fast checks, executed before any processing starts."""
    if not 0 <= config.processing.minimum_grade <= 100:
        raise ConfigurationError("minimum_grade must be between 0 and 100")

    if config.logging.level not in LOG_LEVELS:
        raise ConfigurationError(f"Unknown logging level: {config.logging.level}")

    if not config.input_path.is_file():
        raise ConfigurationError(f"Input file not found: {config.input_path}")

    if config.output_path.is_dir():
        raise ConfigurationError(f"Output path is a directory:
{config.output_path}")



def load_config(
    path: Path, environ: Mapping[str, str] | None = None
) -> AppConfig:
    config = build_config(_read_yaml(path), environ)
    validate_config(config)
    return config
```

```
"""Domain and configuration models."""

from dataclasses import asdict, dataclass
from pathlib import Path



@dataclass(frozen=True, slots=True)
class Student:
    student_id: int
    first_name: str
    last_name: str
    group: str
    average_grade: float

    def to_dict(self) -> dict[str, object]:
        return asdict(self)



@dataclass(frozen=True, slots=True)
class ProcessingConfig:
    minimum_grade: float
    skip_invalid: bool



@dataclass(frozen=True, slots=True)
class LoggingConfig:
    level: str
    path: Path



@dataclass(frozen=True, slots=True)
class AppConfig:
    input_path: Path
    output_path: Path
```

```
    errors_path: Path | None
    processing: ProcessingConfig
    logging: LoggingConfig
```

9. Logging

Налаштування logging винесено в logging_config.py: повідомлення пишуться і в консоль, і у файл із ротацією (RotatingFileHandler, до 1 МБ і 3 резервні копії). Формат повідомлення містить час, рівень, ім’я logger та текст. Некоректні записи реєструються на рівні WARNING з номером рядка, початок і завершення операції — INFO, збої — ERROR; у випадку винятку logger.exception додає traceback. У log не записуються паролі, токени та інші конфіденційні дані: у повідомленнях лише номери рядків і причини помилок.

```
"""Logging setup with file rotation."""

import logging
from logging.handlers import RotatingFileHandler
from pathlib import Path



def configure_logging(
    level: str,
    path: Path,
    *,
    max_bytes: int = 1_000_000,
    backup_count: int = 3,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)

    logging.basicConfig(
        level=getattr(logging, level.upper(), logging.INFO),
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=[
            RotatingFileHandler(
                path,
                maxBytes=max_bytes,
                backupCount=backup_count,
                encoding="utf-8",
            ),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

Приклад log-файла після запуску для data/students.csv у tolerant mode:

```
2026-11-07 10:33:19,957 | INFO | reliable_importer.utils | Student import/export
started
2026-11-07 10:33:19,958 | WARNING | reliable_importer.services | Invalid record at
line 8: first_name is empty
2026-11-07 10:33:19,958 | WARNING | reliable_importer.services | Invalid record at
line 9: average_grade must be between 0 and 100
2026-11-07 10:33:19,958 | WARNING | reliable_importer.services | Invalid record at
line 10: average_grade must be numeric
```

```
2026-11-07 10:33:19,958 | WARNING | reliable_importer.services | Invalid record at
line 11: Invalid student_id
2026-11-07 10:33:19,958 | WARNING | reliable_importer.services | Invalid record at
line 13: group is empty
2026-11-07 10:33:19,958 | INFO | reliable_importer.utils | Student import/export
finished in 0.000725 s
2026-11-07 10:33:19,958 | INFO | reliable_importer.services | Total=14, valid=9,
invalid=5, exported=7
```

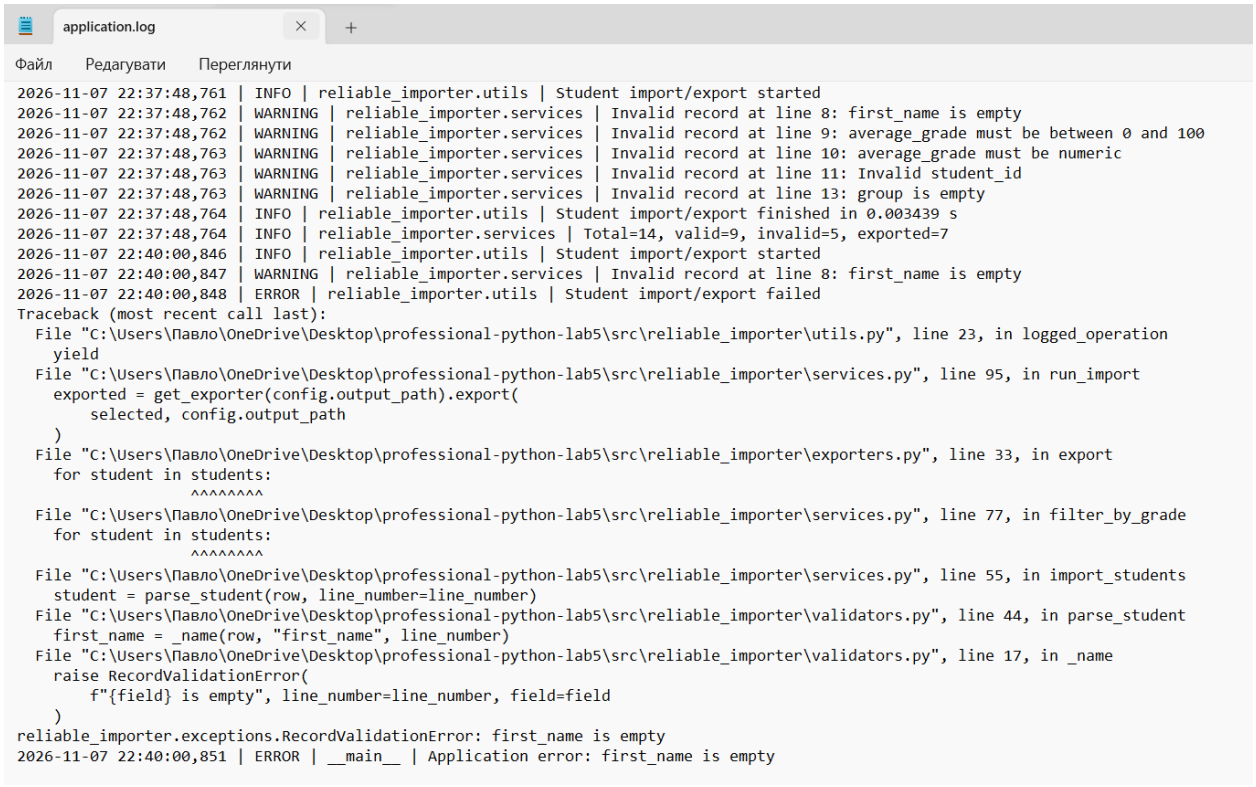


Рис. 2. Фрагмент logs/application.log

10. Streaming reader, validation та статистика

10.1. Streaming reader

read_csv_rows — generator: файл відкривається через with, рядки віддаються по одному разом із номером (нумерація з 2, бо перший рядок — заголовок), тому файл ніколи не завантажується в пам’ять цілком. Перевірка заголовка виконується fail-fast: якщо бракує обов’язкових колонок, одразу піднімається DataImportError. Помилки OSError, UnicodeDecodeError та csv.Error перетворюються на DataImportError. Importer (Protocol) має реалізації CsvImporter та JsonLinesImporter; get_importer вибирає реалізацію за розширенням.

```
"""Streaming readers (importers): CSV and JSON Lines."""


import csv
import json
from collections.abc import Iterator
```

```
from pathlib import Path
from typing import Protocol


from reliable_importer.exceptions import DataImportError


REQUIRED_COLUMNS = ("student_id", "first_name", "last_name", "group",
"average_grade")


Row = dict[str, str]



class Importer(Protocol):
    """Common interface of all importers: yields (line number, row)."""


    def read(self, path: Path) -> Iterator[tuple[int, Row]]: ...



def _check_columns(columns: list[str] | None, path: Path) -> None:
    missing = [c for c in REQUIRED_COLUMNS if not columns or c not in columns]
    if missing:
        raise DataImportError(f"Missing columns in {path}: {', '.join(missing)}")



def read_csv_rows(path: Path) -> Iterator[tuple[int, Row]]:
    """Stream rows of a CSV file one by one; the file is never read whole."""
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            _check_columns(reader.fieldnames and list(reader.fieldnames), path)


            for line_number, row in enumerate(reader, start=2):
                yield line_number, {k: v or "" for k, v in row.items() if k is not
None}
    except (OSError, UnicodeDecodeError, csv.Error) as error:
        raise DataImportError(f"Cannot read input file: {path}") from error



def read_jsonl_rows(path: Path) -> Iterator[tuple[int, Row]]:
    """Stream a JSON Lines file: one JSON object per line."""
    try:
        with path.open("r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                if not line.strip():
                    continue
                try:
                    obj = json.loads(line)
                except json.JSONDecodeError as error:
                    raise DataImportError(
                        f"Invalid JSON at line {line_number} of {path}"
                    ) from error
                if not isinstance(obj, dict):
                    raise DataImportError(f"Line {line_number} is not a JSON
object")
                yield line_number, {
                    str(k): "" if v is None else str(v) for k, v in obj.items()
                }
    except (OSError, UnicodeDecodeError) as error:
        raise DataImportError(f"Cannot read input file: {path}") from error
```

```
class CsvImporter:
    def read(self, path: Path) -> Iterator[tuple[int, Row]]:
        return read_csv_rows(path)



class JsonLinesImporter:
    def read(self, path: Path) -> Iterator[tuple[int, Row]]:
        return read_jsonl_rows(path)



def get_importer(path: Path) -> Importer:
    """Choose an importer by file extension."""
    suffix = path.suffix.lower()
    if suffix == ".csv":
        return CsvImporter()
    if suffix in (".jsonl", ".ndjson"):
        return JsonLinesImporter()
    raise DataImportError(f"Unsupported input format: {suffix or path.name}")
```

10.2. Validation

```
"""Validation of one record; low-level errors become RecordValidationError."""

import math
import re

from reliable_importer.exceptions import RecordValidationError
from reliable_importer.models import Student

NAME_PATTERN = re.compile(r"^[^\W\d_]+(?:[-' ][^\W\d_]+)*$")
GROUP_PATTERN = re.compile(r"^\S{1,20}$")



def _name(row: dict[str, str], field: str, line_number: int) -> str:
    value = (row.get(field) or "").strip()

    if not value:
        raise RecordValidationError(
            f"{field} is empty", line_number=line_number, field=field
        )

    if not NAME_PATTERN.match(value):
        raise RecordValidationError(
            f"{field} contains invalid characters: {value!r}",
            line_number=line_number,
            field=field,
        )

    return value



def parse_student(row: dict[str, str], *, line_number: int) -> Student:
    try:
        student_id = int(row["student_id"])
    except (KeyError, TypeError, ValueError) as error:
        raise RecordValidationError(
            "Invalid student_id", line_number=line_number, field="student_id"
        ) from error
```

```
    if student_id <= 0:
        raise RecordValidationError(
            "student_id must be positive", line_number=line_number,
field="student_id"
        )

    first_name = _name(row, "first_name", line_number)
    last_name = _name(row, "last_name", line_number)

    group = (row.get("group") or "").strip()
    if not group:
        raise RecordValidationError(
            "group is empty", line_number=line_number, field="group"
        )
    if not GROUP_PATTERN.match(group):
        raise RecordValidationError(
            f"Invalid group: {group!r}", line_number=line_number, field="group"
        )

    try:
        grade = float(row["average_grade"].replace(",", "."))
    except (KeyError, TypeError, ValueError, AttributeError) as error:
        raise RecordValidationError(
            "average_grade must be numeric",
            line_number=line_number,
            field="average_grade",
        ) from error

    if not math.isfinite(grade) or not 0 <= grade <= 100:
        raise RecordValidationError(
            "average_grade must be between 0 and 100",
            line_number=line_number,
            field="average_grade",
        )

    return Student(student_id, first_name, last_name, group, grade)
```

10.3. Strict та tolerant processing, statistics

import_students — streaming generator, що створює валідні об’єкти Student по одному. Порушення правил обробляється відповідно до політики: у tolerant mode запис логується, лічильник invalid збільшується (також ведеться розподіл за полями), запис передається в error report і пропускається; у strict mode помилка піднімається повторно (raise) і завершує import. run_import об’єднує все в один use case і повертає RunReport із статистикою Total / valid / invalid / exported.

```
"""Import pipeline: read -> validate -> filter -> export."""

import logging
from collections.abc import Callable, Iterable, Iterator
from dataclasses import dataclass, field
from pathlib import Path
```

```
from reliable_importer.exceptions import RecordValidationError
from reliable_importer.exporters import ErrorReport, get_exporter
from reliable_importer.models import AppConfig, Student
from reliable_importer.readers import get_importer
from reliable_importer.utils import logged_operation
from reliable_importer.validators import parse_student

logger = logging.getLogger(__name__)



@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0
    invalid_by_field: dict[str, int] = field(default_factory=dict)



@dataclass(frozen=True)
class RunReport:
    statistics: ImportStatistics
    exported: int

    def summary(self) -> str:
        s = self.statistics
        return (
            f"Total={s.total}, valid={s.valid}, invalid={s.invalid}, "
            f"exported={self.exported}"
        )



def import_students(
    path: Path,
    *,
    skip_invalid: bool,
    statistics: ImportStatistics,
    on_error: Callable[[RecordValidationError], None] | None = None,
) -> Iterator[Student]:
    """Streaming import: valid objects are created one by one.

    Tolerant mode (skip_invalid=True) logs and skips an invalid record,
    strict mode re-raises the first validation error.
    """
    for line_number, row in get_importer(path).read(path):
        statistics.total += 1


        try:
            student = parse_student(row, line_number=line_number)
        except RecordValidationError as error:
            statistics.invalid += 1
            key = error.field or "unknown"
            statistics.invalid_by_field[key] =
statistics.invalid_by_field.get(key, 0) + 1
            logger.warning("Invalid record at line %s: %s", error.line_number,
error)


            if on_error is not None:
                on_error(error)


            if skip_invalid:
                continue
```

```
            raise

        statistics.valid += 1
        yield student



def filter_by_grade(
    students: Iterable[Student], minimum_grade: float
) -> Iterator[Student]:
    for student in students:
        if student.average_grade >= minimum_grade:
            yield student



def run_import(config: AppConfig) -> RunReport:
    """Whole use case; in strict mode an error produces no output at all."""
    statistics = ImportStatistics()

    with logged_operation("Student import/export"):
        with ErrorReport(config.errors_path) as errors:
            students = import_students(
                config.input_path,
                skip_invalid=config.processing.skip_invalid,
                statistics=statistics,
                on_error=errors.add,
            )
            selected = filter_by_grade(students, config.processing.minimum_grade)
            exported = get_exporter(config.output_path).export(
                selected, config.output_path
            )

    report = RunReport(statistics, exported)
    logger.info(report.summary())
    return report
```

11. Експорт, serialization та atomic output

JsonExporter записує JSON-масив потоково: кожен запис серіалізується окремо (json.dumps з ensure_ascii=False, щоб зберегти кирилицю) і одразу записується у файл. JsonLinesExporter записує по одному об’єкту в рядку. Запис виконується через atomic_writer: дані спочатку потрапляють у тимчасовий файл <name>.tmp, і лише після успішного завершення блоку він замінює справжній файл методом replace. У разі будь-якої помилки тимчасовий файл видаляється, а попередній результат залишається цілим (опційно перед заміною старий файл копіюється в <name>.bak). Помилки запису перетворюються на DataExportError.

```
"""Exporters: atomic JSON / JSON Lines output and the error report."""


import csv
import json
```

```
import textwrap
from collections.abc import Iterable
from contextlib import ExitStack
from pathlib import Path
from types import TracebackType
from typing import Any, Protocol

from reliable_importer.exceptions import DataExportError, RecordValidationError
from reliable_importer.models import Student
from reliable_importer.utils import atomic_writer



class Exporter(Protocol):
    def export(self, students: Iterable[Student], path: Path) -> int: ...



class JsonExporter:
    """Streaming JSON array: records are written one by one."""

    def __init__(self, *, backup: bool = False) -> None:
        self._backup = backup

    def export(self, students: Iterable[Student], path: Path) -> int:
        count = 0

        try:
            with atomic_writer(path, backup=self._backup) as file:
                file.write("[\n")
                for student in students:
                    if count:
                        file.write(",\n")
                    text = json.dumps(student.to_dict(), ensure_ascii=False,
indent=2)
                    file.write(textwrap.indent(text, "  "))
                    count += 1
                file.write("\n]\n")
        except TypeError as error:
            raise DataExportError(f"Data cannot be serialized to JSON: {path}")
from error

        return count



class JsonLinesExporter:
    """JSON Lines: one object per line, convenient for huge outputs."""

    def __init__(self, *, backup: bool = False) -> None:
        self._backup = backup

    def export(self, students: Iterable[Student], path: Path) -> int:
        count = 0

        try:
            with atomic_writer(path, backup=self._backup) as file:
                for student in students:
                    file.write(json.dumps(student.to_dict(), ensure_ascii=False) +
"\n")
                    count += 1
        except TypeError as error:
```

```
            raise DataExportError(f"Data cannot be serialized to JSON: {path}")
from error

        return count



def get_exporter(path: Path, *, backup: bool = False) -> Exporter:
    """Choose an exporter by file extension."""
    suffix = path.suffix.lower()
    if suffix == ".json":
        return JsonExporter(backup=backup)
    if suffix in (".jsonl", ".ndjson"):
        return JsonLinesExporter(backup=backup)
    raise DataExportError(f"Unsupported output format: {suffix or path.name}")



class ErrorReport:
    """Context manager writing invalid records to a separate CSV file.

    ``None`` as a path disables the report. The file is written atomically,
    so a failed run leaves no partial report.
    """

    def __init__(self, path: Path | None) -> None:
        self._path = path
        self._stack: ExitStack | None = None
        self._writer: Any = None

    def __enter__(self) -> "ErrorReport":
        if self._path is not None:
            self._stack = ExitStack()
            file = self._stack.enter_context(atomic_writer(self._path))
            self._writer = csv.writer(file)
            self._writer.writerow(["line_number", "field", "message"])
        return self

    def add(self, error: RecordValidationError) -> None:
        if self._writer is not None:
            self._writer.writerow([error.line_number, error.field, str(error)])

    def __exit__(
        self,
        exc_type: type[BaseException] | None,
        exc_value: BaseException | None,
        traceback: TracebackType | None,
    ) -> bool:
        if self._stack is not None:
            return bool(self._stack.__exit__(exc_type, exc_value, traceback))
        return False
```

Приклад output/students.json (фрагмент):

```
[
  {
    "student_id": 1,
    "first_name": "Pavlo",
    "last_name": "Tsiastun",
    "group": "FeP-33s",
    "average_grade": 93.4
  },
```

```
  {
    "student_id": 2,
    "first_name": "Olena",
    "last_name": "Savchuk",
    "group": "FeP-33s",
    "average_grade": 87.1
  }
]
```

12. Власні context managers

Реалізовано три context managers: logged_operation (generator-based, @contextmanager) логує початок, тривалість і збій операції; atomic_writer (generator-based) забезпечує transaction-like запис файла та очищення тимчасового файла; ErrorReport — клас із методами __enter__ та __exit__, який записує некоректні записи у CSV-звіт і, як і atomic_writer, не залишає частково записаного файла у разі збою. Функція _discard гарантує, що невдале видалення тимчасового файла не приховає початкову помилку.

```
"""Own context managers."""

import logging
import shutil
from collections.abc import Iterator
from contextlib import contextmanager, suppress
from pathlib import Path
from time import perf_counter
from typing import TextIO

from reliable_importer.exceptions import DataExportError

logger = logging.getLogger(__name__)



@contextmanager
def logged_operation(operation: str) -> Iterator[None]:
    """Log start, duration and failure of an operation."""
    logger.info("%s started", operation)
    start = perf_counter()

    try:
        yield
    except Exception:
        logger.exception("%s failed", operation)
        raise
    else:
        logger.info("%s finished in %.6f s", operation, perf_counter() - start)



def _discard(path: Path) -> None:
    """Remove a temporary file; a failed cleanup must not hide the real error."""
    with suppress(OSError):
        path.unlink(missing_ok=True)
```

```
@contextmanager
def atomic_writer(path: Path, *, backup: bool = False) -> Iterator[TextIO]:
    """Transaction-like file write.

    Data is written to ``<name>.tmp``. Only if the block finishes without
    an error, the temporary file replaces the real one (optionally the old
    file is saved as ``<name>.bak``). Otherwise the temporary file is removed
    and the previous output stays untouched.
    """
    temporary = path.with_name(path.name + ".tmp")

    try:
        path.parent.mkdir(parents=True, exist_ok=True)

        with temporary.open("w", encoding="utf-8", newline="") as file:
            yield file

        if backup and path.exists():
            shutil.copy2(path, path.with_name(path.name + ".bak"))

        temporary.replace(path)
    except OSError as error:
        _discard(temporary)
        raise DataExportError(f"Cannot write {path}") from error
    except BaseException:
        _discard(temporary)
        raise
```

13. Точка входу та результат роботи

main.py читає параметр --config, завантажує конфігурацію, налаштовує logging та запускає run_import. Очікувані помилки (ApplicationError) логуються, і програма завершується з кодом 1; неочікувані — з traceback у log і кодом 2; успішне завершення має код 0. Таким чином application коректно завершується при critical errors і не показує користувачу сирого traceback для передбачуваних ситуацій.

```
"""Application entry point."""

import argparse
import logging
from collections.abc import Sequence
from pathlib import Path

from reliable_importer.config import load_config
from reliable_importer.exceptions import ApplicationError
from reliable_importer.logging_config import configure_logging
from reliable_importer.services import run_import

logger = logging.getLogger(__name__)



def main(argv: Sequence[str] | None = None) -> None:
```

```
    parser = argparse.ArgumentParser(description="Reliable student data importer")
    parser.add_argument("--config", type=Path, default=Path("config.yaml"))
    args = parser.parse_args(argv)

    try:
        config = load_config(args.config)
        configure_logging(config.logging.level, config.logging.path)
        run_import(config)
    except ApplicationError as error:
        logger.error("Application error: %s", error)
        raise SystemExit(1) from error
    except Exception as error:
        logger.exception("Unexpected failure")
        raise SystemExit(2) from error



if __name__ == "__main__":
    main()
```

Запуск: python -m reliable_importer.main.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab5> python -m reliable_importer.main
2026-11-07 22:47:11,184 | INFO | reliable_importer.utils | Student import/export started
2026-11-07 22:47:11,186 | WARNING | reliable_importer.services | Invalid record at line 8: first_name is empty
2026-11-07 22:47:11,186 | WARNING | reliable_importer.services | Invalid record at line 9: average_grade must be between 0 and 100
2026-11-07 22:47:11,186 | WARNING | reliable_importer.services | Invalid record at line 10: average_grade must be numeric
2026-11-07 22:47:11,186 | WARNING | reliable_importer.services | Invalid record at line 11: Invalid student_id
2026-11-07 22:47:11,186 | WARNING | reliable_importer.services | Invalid record at line 13: group is empty
2026-11-07 22:47:11,188 | INFO | reliable_importer.utils | Student import/export finished in 0.003583 s
2026-11-07 22:47:11,188 | INFO | reliable_importer.services | Total=14, valid=9, invalid=5, exported=7
```

Рис. 3. Результат запуску python -m reliable_importer.main

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab5> python -m unittest discover -s tests -v
test_environment_override (test_importer.ConfigTest.test_environment_override) ... ok
test_invalid_structures (test_importer.ConfigTest.test_invalid_structures) ... ok
test_load_errors (test_importer.ConfigTest.test_load_errors) ... ok
test_semantic_validation (test_importer.ConfigTest.test_semantic_validation) ... ok
test_valid_configuration (test_importer.ConfigTest.test_valid_configuration) ... ok
test_atomic_writer_commits_only_on_success (test_importer.ContextManagerTest.test_atomic_writer_commits_only_on_success) ... ok
test_logged_operation_logs_success_and_failure (test_importer.ContextManagerTest.test_logged_operation_logs_success_and_failure) ... ok
test_hierarchy (test_importer.ExceptionTest.test_hierarchy) ... ok
test_record_error_carries_context (test_importer.ExceptionTest.test_record_error_carries_context) ... ok
test_atomic_failure_keeps_previous_output (test_importer.ExportTest.test_atomic_failure_keeps_previous_output) ... ok
test_backup_of_existing_output (test_importer.ExportTest.test_backup_of_existing_output) ... ok
test_empty_export_is_valid_json (test_importer.ExportTest.test_empty_export_is_valid_json) ... ok
test_error_report (test_importer.ExportTest.test_error_report) ... ok
test_error_report_is_removed_on_failure (test_importer.ExportTest.test_error_report_is_removed_on_failure) ... ok
test_exporter_selected_by_extension (test_importer.ExportTest.test_exporter_selected_by_extension) ... ok
test_json_export_is_valid_and_keeps_unicode (test_importer.ExportTest.test_json_export_is_valid_and_keeps_unicode) ... ok
test_jsonl_export (test_importer.ExportTest.test_jsonl_export) ... ok
test_write_error_becomes_export_error (test_importer.ExportTest.test_write_error_becomes_export_error) ... ok
test_configuration_error_exits_with_code_1 (test_importer.MainTest.test_configuration_error_exits_with_code_1) ... ok
test_run_import_reports_statistics (test_importer.MainTest.test_run_import_reports_statistics) ... ok
test_strict_run_exits_with_code_1_and_no_output (test_importer.MainTest.test_strict_run_exits_with_code_1_and_no_output) ... ok
test_tolerant_run (test_importer.MainTest.test_tolerant_run) ... ok
test_bad_encoding (test_importer.ReaderTest.test_bad_encoding) ... ok
test_csv_streaming_and_line_numbers (test_importer.ReaderTest.test_csv_streaming_and_line_numbers) ... ok
test_invalid_jsonl_and_unknown_extension (test_importer.ReaderTest.test_invalid_jsonl_and_unknown_extension) ... ok
test_jsonl (test_importer.ReaderTest.test_jsonl) ... ok
test_missing_columns_fail_fast (test_importer.ReaderTest.test_missing_columns_fail_fast) ... ok
test_missing_file (test_importer.ReaderTest.test_missing_file) ... ok
test_filter_by_grade (test_importer.ServiceTest.test_filter_by_grade) ... ok
test_strict_mode_stops_at_first_error (test_importer.ServiceTest.test_strict_mode_stops_at_first_error) ... ok
test_tolerant_mode_counts_and_skips (test_importer.ServiceTest.test_tolerant_mode_counts_and_skips) ... ok
test_decimal_comma (test_importer.ValidatorTest.test_decimal_comma) ... ok
test_exception_chaining_keeps_cause (test_importer.ValidatorTest.test_exception_chaining_keeps_cause) ... ok
test_invalid_fields (test_importer.ValidatorTest.test_invalid_fields) ... ok
test_missing_key_is_validation_error (test_importer.ValidatorTest.test_missing_key_is_validation_error) ... ok
test_non_finite_and_negative (test_importer.ValidatorTest.test_non_finite_and_negative) ... ok
test_valid_record (test_importer.ValidatorTest.test_valid_record) ... ok

----------------------------------------------------------------------
Ran 37 tests in 0.070s

OK
```

Рис. 4. Результат запуску unit-тестів



Рис. 5. Файл output/students.json та output/errors.csv

13.1. Strict mode

Для перевірки strict mode у config.yaml потрібно встановити skip_invalid: false (або передати змінну середовища IMPORTER_SKIP_INVALID=false).

Перша ж некоректна запис (рядок 8) завершує import: у log з’являється ERROR із traceback, програма завершується з кодом 1, а output-файли не створюються.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab5> Remove-Item output\students.json, output\errors.csv -ErrorAction SilentlyContinue
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab5> $env:IMPORTER_SKIP_INVALID="false"
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab5> python -m reliable_importer.main
2026-11-07 22:58:19,474 | INFO | reliable_importer.utils | Student import/export started
2026-11-07 22:58:19,476 | WARNING | reliable_importer.services | Invalid record at line 8: first_name is empty
2026-11-07 22:58:19,477 | ERROR | reliable_importer.utils | Student import/export failed
Traceback (most recent call last):
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\utils.py", line 23, in logged_operation
    yield
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\services.py", line 95, in run_import
    exported = get_exporter(config.output_path).export(
        selected, config.output_path
    )
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\exporters.py", line 33, in export
    for student in students:
                   ^^^^^^^^
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\services.py", line 77, in filter_by_grade
    for student in students:
                   ^^^^^^^^
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\services.py", line 55, in import_students
    student = parse_student(row, line_number=line_number)
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\validators.py", line 44, in parse_student
    first_name = _name(row, "first_name", line_number)
  File "C:\Users\Павло\OneDrive\Desktop\professional-python-lab5\src\reliable_importer\validators.py", line 17, in _name
    raise RecordValidationError(
        f"{field} is empty", line_number=line_number, field=field
    )
reliable_importer.exceptions.RecordValidationError: first_name is empty
2026-11-07 22:58:19,479 | ERROR | __main__ | Application error: first_name is empty
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab5> echo $LASTEXITCODE
1
```

Рис. 6. Strict mode: завершення з кодом 1

14. Експерименти

Усі експерименти виконуються командою python -m reliable_importer.experiments.

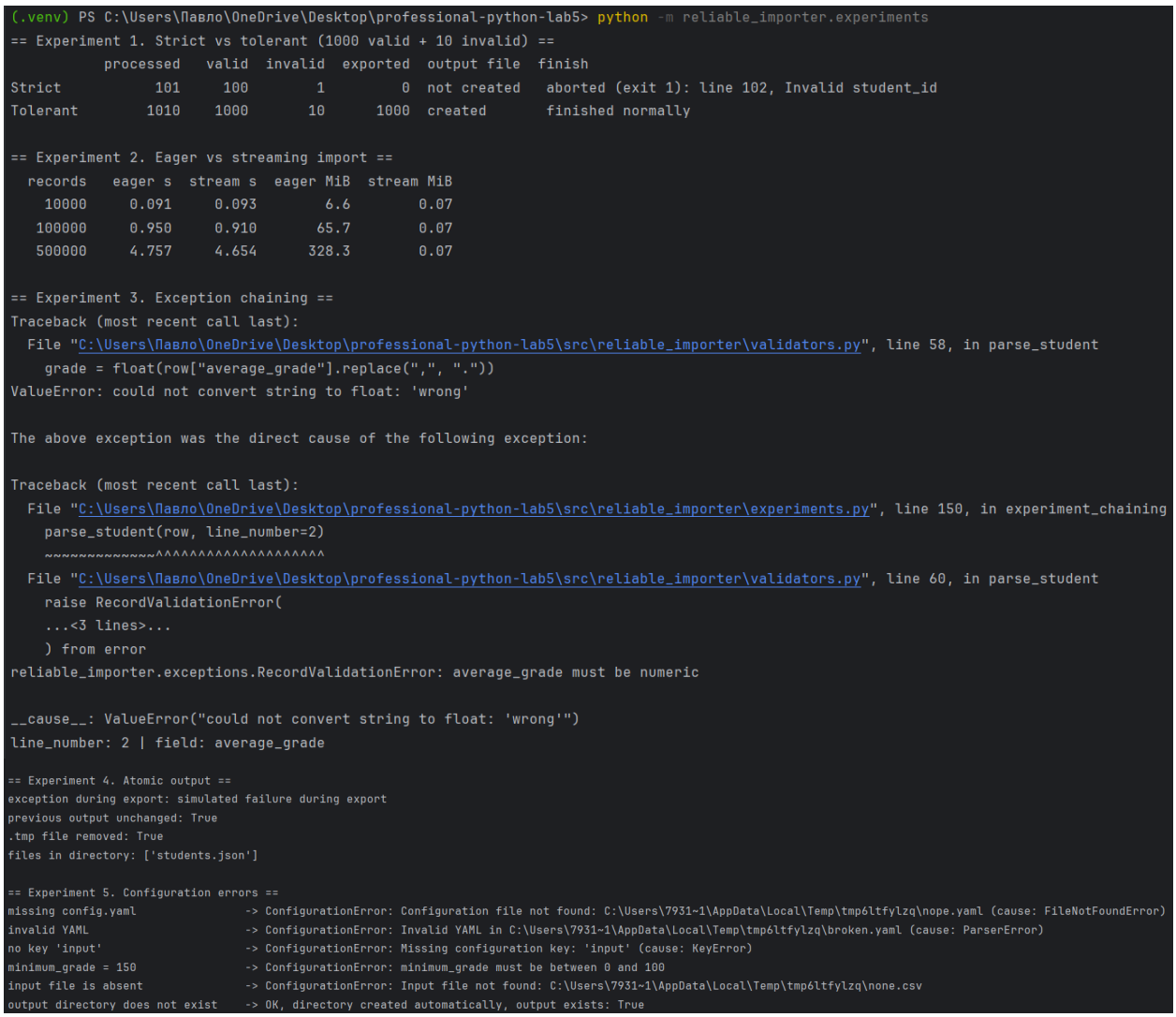


Рис. 7. Результат запуску python -m reliable_importer.experiments

14.1. Strict проти tolerant

Створено CSV із 1000 коректних і 10 некоректних записів (некоректні розподілені рівномірно).

Параметр	Strict	Tolerant
Оброблено записів	101	1010
Valid	100	1000
Invalid	1	10
Exported	0 (файл не створено)	1000
Завершення application	Аварійне, код 1, помилка в рядку 102	Нормальне, код 0

Strict mode доцільний, коли дані мають бути повністю коректними (фінансові записи, критичні довідники, перед завантаженням у production): краще зупинитись і виправити джерело, ніж отримати неповний результат.

Tolerant mode доцільний для великих неідеальних наборів (логи, дані від користувачів, імпорт з різних систем), коли важливо обробити більшість записів, а помилки зареєструвати в log та error report для подальшого розбору.

14.2. Streaming проти eager

Eager-варіант читає весь файл у list, перевіряє записи в іншому list і записує результат одним json.dump. Streaming-варіант пропускає кожен запис через generator pipeline і записує його одразу. Час вимірювався через perf_counter, пік пам'яті — через tracemalloc (pipeline обов'язково споживається повністю).

Records	Eager time, c	Streaming time, c	Eager peak RAM, MiB	Streaming peak RAM, MiB
10 000	0.071	0.131	6.8	0.20
100 000	0.758	1.338	65.7	0.70
500 000	3.744	6.692	328.3	2.98

Висновок: пік пам'яті eager-варіанта зростає пропорційно розміру файла (на 500 000 записів 328 МіБ), а streaming-варіанта лишається в межах кількох МіБ (приблизно у 110 разів менше). Водночас streaming у цьому експерименті повільніший (близько 1,8 раза): кожен запис окремо проходить generator-етапи, створюється об'єкт Student і окремий виклик json.dumps, тоді як eager-варіант використовує один json.dump над готовим списком. Тобто streaming дає суттєву перевагу за пам'яттю, але не гарантує мінімального часу виконання. Абсолютні значення залежать від комп'ютера.

14.3. Exception chaining

Для запису student_id,first_name,last_name,group,average_grade = 1,Anna,Koval,FeP-21,wrong виконано parse_student. Traceback показує ланцюжок (шляхи скорочено):

```
Traceback (most recent call last):
  File "validators.py", line 58, in parse_student
    grade = float(row["average_grade"].replace(",", "."))
ValueError: could not convert string to float: 'wrong'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "validators.py", line 60, in parse_student
    raise RecordValidationError(
reliable_importer.exceptions.RecordValidationError: average_grade must be numeric
```

```
error.__cause__  -> ValueError("could not convert string to float: 'wrong'")
error.line_number -> 2,  error.field -> 'average_grade'
```

Ланцюжок ValueError → RecordValidationError зберігає і технічну причину (що саме не вдалося перетворити), і доменний контекст (номер рядка та поле). Без from error лишилась би лише доменна помилка, і діагностика була б складнішою.

14.4. Atomic output

У файл students.json спочатку записано попередній коректний результат, а потім експорт навмисно перервано винятком RuntimeError після двох записів. Результат:

Перевірка	Результат
Попередній valid output не пошкоджено	так (вміст файла не змінився)
.tmp-файл очищено	так (у каталозі лише students.json)
Exception зареєстровано	так (піднято до рівня виклику; у застосунку його ловить main.py)
Application завершується контрольовано	так (код завершення 2 для неочікуваної помилки; для ApplicationError — код 1)

14.5. Помилки конфігурації

Сценарій	Поведінка application
відсутній config.yaml	ConfigurationError: Configuration file not found (причина FileNotFoundError), код завершення 1
неправильний YAML	ConfigurationError: Invalid YAML in … (причина ParserError)
немає ключа input	ConfigurationError: Missing configuration key: 'input' (причина KeyError)
minimum_grade = 150	ConfigurationError: minimum_grade must be between 0 and 100
input file відсутній	ConfigurationError: Input file not found: …
output directory не існує	Не є помилкою: каталог створюється автоматично (mkdir parents=True), результат записується

Усі помилки конфігурації виявляються на старті (fail-fast), до початку обробки даних, і завершують програму з кодом 1 без створення output-файлів.

15. Git

```
git init
git add .gitignore pyproject.toml config.yaml src/reliable_importer/__init__.py
git commit -m "Create project structure"
git add src/reliable_importer/exceptions.py src/reliable_importer/models.py
src/reliable_importer/config.py src/reliable_importer/logging_config.py
```

```
git commit -m "Add exceptions, models, configuration and logging"
git add src/reliable_importer/readers.py src/reliable_importer/validators.py
src/reliable_importer/utils.py src/reliable_importer/exporters.py
src/reliable_importer/services.py src/reliable_importer/main.py data
git commit -m "Add streaming import, validation and atomic export"
git add src/reliable_importer/experiments.py tests README.md output/.gitkeep
logs/.gitkeep
git commit -m "Add experiments, tests and documentation"
```

```
C:\Users\Павло\OneDrive\Desktop\professional-python-lab5>git status
On branch main
nothing to commit, working tree clean
```

Рис. 8. Результат git status

```
C:\Users\Павло\OneDrive\Desktop\professional-python-lab5>git log --oneline
cfd1fd0 (HEAD -> main) Add experiments, tests and documentation
51a2a68 Add streaming import, validation and atomic export
9f34b18 Add exceptions, models, configuration and logging
a4a4df6 Create project structure
```

Рис. 9. Результат git log --oneline

Репозиторій опубліковано на GitHub: https://github.com/Pavlo12112/professional-python-lab5

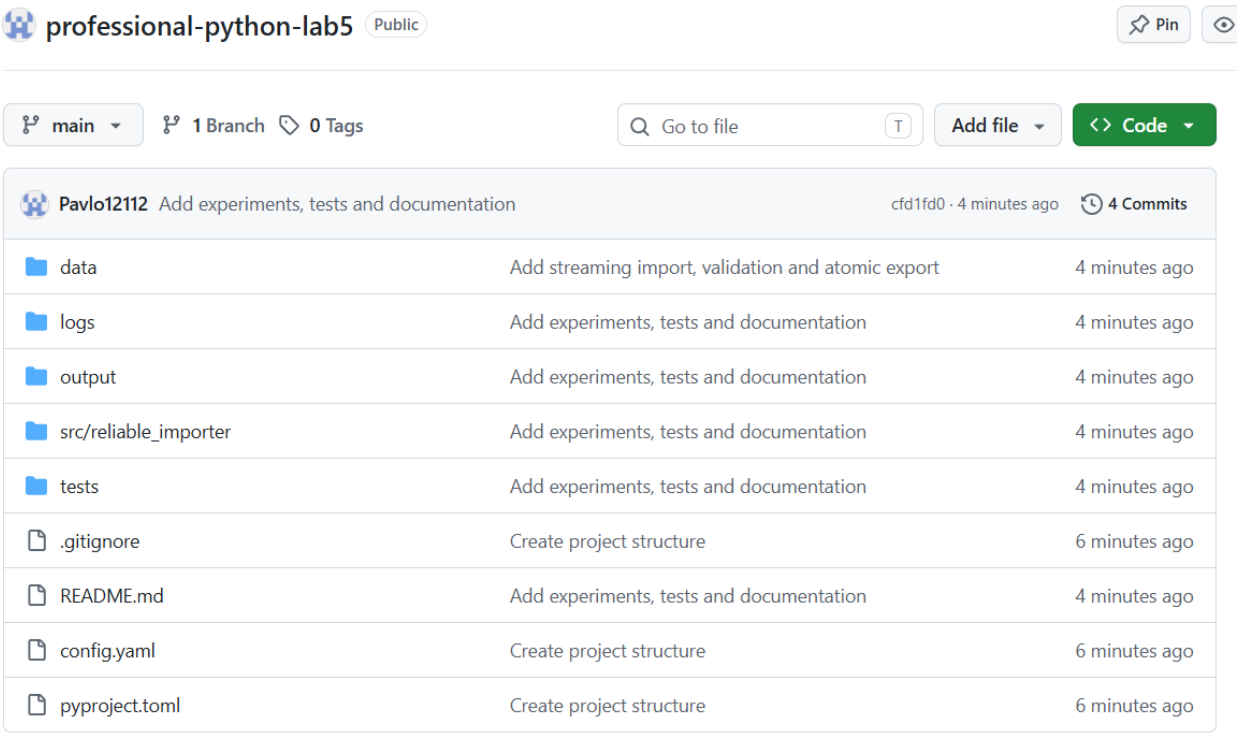


Рис. 10. Репозиторій на GitHub

16. Відповіді на контрольні запитання

1. Що таке exception?

Об’єкт, який сигналізує про ненормальну ситуацію під час виконання (ValueError, FileNotFoundError) і перериває звичайний потік виконання, доки його не буде оброблено.

2. Для чого використовується try?

Позначає блок коду, під час виконання якого можуть виникнути винятки, що потрібно обробити.

3. Для чого except?

Обробляє виняток певного типу, що виник у блоці try.

4. Коли виконується else після try?

Лише якщо у блоці try виняток не виник.

5. Коли виконується finally?

Завжди: і після успіху, і після помилки, тому там звільняють ресурси.

6. Для чого використовується raise?

Щоб навмисно піднята виняток (наприклад, при порушенні правила валідації).

7. Що означає raise без аргументів?

Повторно піднімає поточний виняток у блоці except із збереженням traceback (у services це raise у strict mode).

8. Чому не рекомендується except: pass?

Він мовчки ховає будь-які помилки, включно з програмними, ускладнює налагодження й втрачає причину збою.

9. Чому потрібно перехоплювати конкретні exception types?

Щоб обробляти лише очікувані ситуації й не приховувати інші помилки; кожному типу відповідає своя реакція.

10. Що таке exception hierarchy?

Дерево винятків від спільного базового класу (ApplicationError → DataError → RecordValidationError), що дозволяє перехоплювати як конкретну помилку, так і цілу групу.

11. Для чого створювати custom exceptions?

Щоб помилки мали зміст предметної області, несли додаткові дані (рядок, поле) та відокремлювались від чужих винятків.

12. Що таке exception chaining?

Зв’язування нового винятку з тим, що його спричинив, так що traceback показує обидва.

13. Що означає raise ... from ...?

Створює явний ланцюжок: новий виняток отримує атрибут __cause__ із початковою причиною.

14. Яку перевагу дає chaining для debugging?

Видно першопричину (ValueError) разом із доменною помилкою (RecordValidationError), тож діагностика швидша.

15. Що означає EAFP?

«Easier to Ask Forgiveness than Permission»: спробувати виконати дію й обробити виняток, замість численних попередніх перевірок.

16. Що означає LBYL?

«Look Before You Leap»: спершу перевірити умови (наприклад, key in data), а потім діяти.

17. Що таке fail-fast?

Виявляти некоректні вхідні умови якомога раніше та негайно зупиняти роботу, а не обробляти дані з прихованою помилкою.

18. Де доцільно застосовувати fail-fast?

При перевірці конфігурації на старті (validate_config), заголовка вхідного файла та обов’язкових параметрів до початку довгої обробки.

19. Що таке context manager?

Об’єкт, що керує життєвим циклом ресурсу: гарантує підготовку при вході в with і очищення при виході.

20. Для чого використовується with?

Для гарантованого звільнення ресурсів (закриття файла, повернення блокування) навіть у разі винятку.

21. Які methods утворюють context manager protocol?

__enter__() та __exit__().

22. Що робить __enter__()?

Виконується при вході в with, готує ресурс і повертає об’єкт, який зв’язується з ім’ям після as.

23. Що робить __exit__()?

Виконується при виході з with (також при винятку), звільняє ресурс і може придушити виняток, повернувши True.

24. Чи закриває with файл при exception?

Так, файл закривається незалежно від того, чи виник виняток.

25. Для чого contextlib.contextmanager?

Дозволяє створити context manager простою generator-функцією замість класу з __enter__ і __exit__ (logged_operation, atomic_writer).

26. Яку роль має yield у generator-based context manager?

Розділяє підготовку (код до yield) і очищення (після yield або у finally); на місці yield виконується тіло with.

27. Що таке pathlib.Path?

Об’єктно-орієнтований, кросплатформений інтерфейс для роботи зі шляхами та файлами.

28. Чому pathlib кращий за ручну конкатенацію шляхів?

Оператор / будує шляхи переносимо між ОС, а методи (exists, mkdir, with_name, suffix) замінюють рядкові маніпуляції.

29. Для чого задається encoding="utf-8"?

Щоб кодування не залежало від налаштувань ОС і кирилиця читалась та записувалась однаково скрізь.

30. Що таке CSV?

Текстовий табличний формат, де значення рядка розділені комою (або іншим роздільником).

31. Для чого csv.reader?

Читає CSV-рядки як списки значень.

32. Для чого csv.DictReader?

Читає кожен рядок як словник, ключі якого беруться із заголовка.

33. Чим DictReader зручний для structured data?

Доступ до полів за назвою (row["group"]), а не за номером колонки, тому код читабельніший і стійкіший до зміни порядку колонок.

34. Для чого csv.DictWriter?

Записує словники як рядки CSV за заданим порядком полів (fieldnames) і може записати заголовок.

35. Що таке JSON?

Текстовий формат обміну даними (об’єкти, масиви, рядки, числа, true/false/null), поширений в API, експорті та конфігурації.

36. Що таке serialization?

Перетворення об’єкта чи структури даних у формат для зберігання або передавання (об’єкт → словник → JSON → файл).

37. Що таке deserialization?

Зворотний процес: відновлення структури даних із текстового чи двійкового представлення (файл → JSON → словник → доменний об’єкт).

38. Чим json.dump() відрізняється від json.dumps()?

dump записує JSON у файловий об’єкт, dumps повертає рядок.

39. Чим json.load() відрізняється від json.loads()?

load читає JSON із файлового об’єкта, loads — із рядка.

40. Що таке JSONDecodeError?

Виняток json.JSONDecodeError (підклас ValueError), який виникає при некоректному JSON; його перетворюють на доменну помилку через from.

41. Для чого використовується YAML?

Для зручної для людини конфігурації з вкладеними структурами.

42. Чому рекомендується yaml.safe_load()?

Він створює лише прості типи Python й не виконує довільний код, а yaml.load на недовірених даних небезпечний.

43. Які типові сценарії використання JSON?

REST API, обмін даними між системами, експорт, кеші та машинно-орієнтована конфігурація.

44. Які типові сценарії використання YAML?

Конфігурація, яку редагує людина (CI, Docker Compose, налаштування застосунків).

45. Що таке configuration file?

Файл із параметрами застосунку, які можна змінити без редагування коду (config.yaml).

46. Чому configuration краще не hard-code у Python?

Зміна поведінки (поріг, режим, шляхи) тоді не потребує змін коду й перезбирання, а середовища (розробка, production) мають різні значення.

47. Що потрібно перевіряти у configuration?

Наявність обов'язкових ключів, типи та діапазони значень, існування вхідних файлів, допустимі рівні logging.

48. Що таке logging?

Стандартний модуль для запису подій застосунку з рівнями, форматом та різними виводами (консоль, файл).

49. Чим logging відрізняється від print()?

Має рівні важливості, форматування з часом, кілька обробників, налаштування без зміни коду та запис у файли з ротацією.

50. Які існують logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL.

51. Для чого logger.info()?

Для повідомлень про нормальний перебіг (початок, завершення, статистика).

52. Для чого logger.warning()?

Для небезпечних, але не фатальних ситуацій (пропущений некоректний запис).

53. Для чого logger.error()?

Для помилок, через які операція не виконана (збій імпорту, помилка застосунку).

54. Для чого logger.exception()?

У блоці except записує повідомлення рівня ERROR разом із traceback.

55. Що містить traceback?

Стек викликів у момент помилки: файли, рядки, функції та тип і текст винятку (а при chaining — і причину).

56. Чому secrets не можна записувати в logs?

Логи зберігаються довго та доступні багатьом, тому паролі, токени й API-ключі можуть бути викрадені.

57. Що таке streaming I/O?

Читання й запис даних частинами (по рядку чи записі) без повного завантаження файла в пам’ять.

58. Яка перевага streaming для великих files?

Пам’ять майже не залежить від розміру файла, тому можна обробляти набори, більші за RAM; результат з’являється поступово.

59. Що таке eager file loading?

Повне завантаження вмісту файла в пам’ять одразу (read(), list(reader)).

60. Чому list(reader) може бути небажаним?

Для великого файла він зберігає всі рядки в пам’яті одночасно, що показав експеримент (328 МіБ проти 3 МіБ на 500 000 записів).

61. Що таке strict validation mode?

Режим, де перший некоректний запис перериває обробку (виняток).

62. Що таке tolerant mode?

Режим, де некоректний запис логується, лічиться й пропускається, а обробка триває.

63. Коли доцільно використовувати strict mode?

Коли дані мають бути повністю коректними і частковий результат неприпустимий (фінанси, критичні дані).

64. Коли tolerant mode?

Для великих «брудних» наборів, де важливо обробити більшість записів і розібрати помилки пізніше.

65. Що таке atomic write?

Запис, який або повністю завершується, або взагалі не змінює результат: тимчасовий файл → успіх → replace.

66. Для чого використовується temporary file?

Щоб писати дані не в справжній output, а в проміжний файл, який замінить його лише після успішного завершення.

67. Що може статися при звичайному output write та аварійному завершенні?

Файл залишиться частково записаним (пошкоджений JSON), а попередній коректний результат буде втрачено.

68. Навіщо видаляти temporary file після error?

Щоб не накопичувати сміття й не сплутати неповний файл із результатом.

69. Що таке JSON Lines?

Формат, де кожен рядок файла — окремий JSON-об’єкт.

70. Чому JSON Lines зручний для streaming?

Кожен об’єкт можна прочитати та записати окремим рядком без розбору всього документа, а файл легко дописувати.

17. Висновок

У лабораторній роботі створено надійний модуль імпорту, експорту та конфігурації даних студентів (варіант 1). Реалізовано власну ієрархію exceptions із exception chaining, fail-fast валідацію YAML-конфігурації, streaming-читання CSV із валідацією кожного запису, режими strict та tolerant зі статистикою valid/invalid, logging із ротацією, власні context managers та atomic export у JSON без ризику пошкодити попередній результат.

Експерименти підтвердили: strict mode зупиняє обробку на першій помилці й не створює вихідних файлів, tolerant mode обробляє все коректне; streaming-імпорт використовує у ~110 разів менше пам’яті (3 МіБ проти 328 МіБ на 500 000 записів), хоча не завжди швидший; ланцюжок винятків зберігає першопричину; atomic write залишає попередній результат цілим, а помилки конфігурації виявляються на старті. Отриманий модуль можна використовувати як інфраструктурну частину професійного Python-проєкту.